



US012407855B2

(12) **United States Patent**
Krishna et al.

(10) **Patent No.:** **US 12,407,855 B2**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **IMPLICIT MODE DEPENDENT PRIMARY TRANSFORMS**

(71) Applicant: **TENCENT AMERICA LLC**, Palo Alto, CA (US)

(72) Inventors: **Madhu Peringassery Krishna**, Mountain View, CA (US); **Xin Zhao**, San Diego, CA (US); **Shan Liu**, San Jose, CA (US)

(73) Assignee: **TENCENT AMERICA LLC**, Palo Alto, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **18/239,292**

(22) Filed: **Aug. 29, 2023**

(65) **Prior Publication Data**

US 2023/0403410 A1 Dec. 14, 2023

Related U.S. Application Data

(63) Continuation of application No. 17/066,791, filed on Oct. 9, 2020, now Pat. No. 11,785,254.

(60) Provisional application No. 63/032,216, filed on May 29, 2020.

(51) **Int. Cl.**

H04N 19/593 (2014.01)

H04N 19/159 (2014.01)

H04N 19/176 (2014.01)

H04N 19/70 (2014.01)

(52) **U.S. Cl.**

CPC **H04N 19/593** (2014.11); **H04N 19/159** (2014.11); **H04N 19/176** (2014.11); **H04N 19/70** (2014.11)

(58) **Field of Classification Search**

CPC .. H04N 19/593; H04N 19/159; H04N 19/176; H04N 19/70; H04N 19/157; H04N 19/46; H04N 19/12; H04N 19/11; H04N 19/186
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

9,674,530 B1	6/2017	Bultje et al.	
10,432,929 B2	10/2019	Zhao et al.	
10,567,763 B2 *	2/2020	Koo	H04N 19/46
2013/0128977 A1 *	5/2013	Yu	H04N 19/463 375/240.12

(Continued)

FOREIGN PATENT DOCUMENTS

WO	2020/046084 A1	3/2020
WO	2020/058568 A1	3/2020

OTHER PUBLICATIONS

Kim W.S. "3-D Video Coding System with Enhanced Rendered View Quality" Dissertation, University of Southern California, Aug. 2011 (Year: 2011).*

(Continued)

Primary Examiner — Richard A Hansell, Jr.

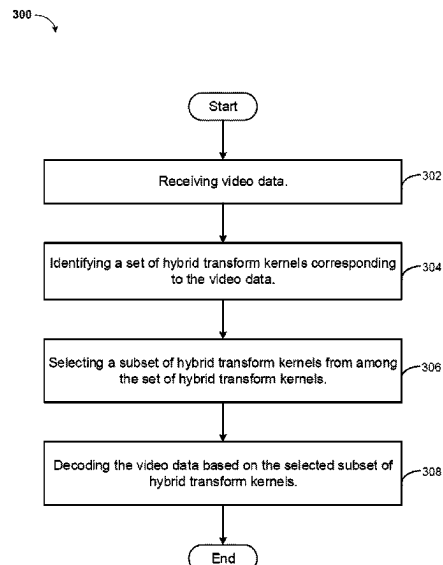
(74) *Attorney, Agent, or Firm* — Sughrue Mion, PLLC

(57)

ABSTRACT

A method, computer program, and computer system is provided for coding video data. Video data is received, and a set of hybrid transform kernels corresponding to the video data is identified. A subset of hybrid transform kernels is selected, either explicitly or implicitly, from among the set of hybrid transform kernels. The video data is decoded based on the selected subset of hybrid transform kernels.

14 Claims, 6 Drawing Sheets



(56)

References Cited**U.S. PATENT DOCUMENTS**

2015/0145874	A1	5/2015	Han et al.	
2017/0094314	A1	3/2017	Zhao et al.	
2018/0020218	A1	1/2018	Zhao et al.	
2018/0041760	A1 *	2/2018	Koo	H04N 19/122
2018/0115787	A1	4/2018	Koo et al.	
2018/0146195	A1 *	5/2018	Koo	H04N 19/61
2018/0220157	A1 *	8/2018	Koo	H04N 19/60
2018/0220158	A1 *	8/2018	Koo	H04N 19/159
2018/0249179	A1	8/2018	Han et al.	
2019/0028738	A1	1/2019	Philippe et al.	
2019/0052876	A1 *	2/2019	Lee	H04N 19/157
2019/0215516	A1 *	7/2019	Lim	H04N 19/107
2019/0281299	A1 *	9/2019	Leleannec	H04N 19/61
2019/0387241	A1 *	12/2019	Kim	H04N 19/147
2020/0288129	A1 *	9/2020	Egilmez	H04N 19/18
2020/0374554	A1 *	11/2020	Koo	H04N 19/12
2021/0195192	A1 *	6/2021	Coban	H04N 19/176
2022/0053189	A1 *	2/2022	Koo	H04N 19/70

OTHER PUBLICATIONS

De Rivaz and Haughton "AV1 Bitstream & Decoding Process Specification", Jan. 8, 2019 (Year: 2019).*

Bross et al., "General Video Coding Technology in Responses to the Joint Call for Proposals on Video Compression with Capability beyond HEVC", IEEE Transactions on Circuits and Systems for Video Technology, 2019, pp. 1-16 (16 pages total).

Bross et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 11th Meeting: Ljubljana, SI, Jul. 10-18, 2018, JVET-K1001-v6 (141 pages total).

Bross et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 12th Meeting: Macao, CN, Oct. 3-12, 2018, JVET-L0285-r1 (7 pages total).

Bross et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, JVET-O2001-vE (455 pages total).

Chang et al., "Intra prediction using multiple reference lines for the versatile video coding standard", Proc. SPIE 11137, Applications of Digital Image Processing XLII, 1113716 (Sep. 6, 2019) (8 pages total).

Chen et al., "Screen Content Coding Using Non-Square Intra Block Copy for HEVC", 2014 IEEE International Conference on Multimedia and Expo (ICME), Chengdu, 2014, pp. 1-6 (6 pages total).

Chuohao Yeo et al., "Mode-Dependent Transforms for Coding Directional Intra Prediction Residuals", IEEE Transactions on Circuits and Systems for Video Technology, Apr. 2012, vol. 22, No. 4, pp. 545-554 (10 pages).

De Rivaz et al., "AV1 Bitstream & Decoding Process Specification", Version 1.0.0 with Errata 1, 2018, The Alliance for Open Media (681 pages total).

Extended European Search Report issued Mar. 2, 2023 in European Application No. 21813788.3.

Guo et al., "Inter-layer Adaptive Filtering for Scalable Extension of HEVC", 2013 Picture Coding Symposium (PCS), San Jose, CA, 2013, pp. 165-168 (4 pages total).

Guo et al., "Inter-layer Intra Mode Prediction for Scalable Extension of HEVC", 2013 Picture Coding Symposium (PCS), San Jose, CA, 2013, pp. 317-320 (4 pages total).

Hilmi E. Egilmez et al., "AHG17/Non-CE6: High-level syntax for MTS and Implicit Transform Derivations", Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, JVET-P0569-V1, Oct. 1-11, 2019, 16th Meeting, Geneva, CH (10 pages total).

International Search Report dated Apr. 26, 2021 in International Application No. PCT/US2021/016203.

Japanese Office Action dated Apr. 11, 2023 in Japanese Application No. 2022-526721.

Jianle Chen et al., "Algorithm Description of Joint Exploration Test Model 7 (JEM7)", Joint Video Exploration Team (JVET) of ITU-T

SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, JVET-G1001-v1, 7th Meeting: Torino, IT, 2017, pp. 28-30 (7 pages total).

Lai et al., "Combined Temporal and Inter-layer Prediction for Scalable Video Coding using HEVC", 2013 Picture Coding Symposium (PCS), San Jose, CA, 2013, pp. 117-120 (4 pages total).

Lai et al., "Low Latency Directional Filtering for Inter-layer Prediction in Scalable Video Coding using HEVC", 2013 Picture Coding Symposium (PCS), San Jose, CA, 2013, pp. 269-272 (4 pages total).

Liu et al., "Bit Allocation for Video Coding with Temporal-Spatial Tradeoff", Advances in Multimedia Information Processing—PCM 2001. PCM 2001. Lecture Notes in Computer Science, vol. 2195. Springer, Berlin, Heidelberg, pp. 466-473 (10 pages total).

Liu et al., "Bit-depth Scalable Coding for High Dynamic Range Video", Proc. SPIE 6822, Visual Communications and Image Processing 2008, 68220O (Jan. 28, 2008) (12 pages total).

Liu et al., "Complexity Reduction of Joint Temporal-Spatial Bit Allocation Using R-D Models for Video Streaming", IEEE ICIP 2002, pp. 729-732 (4 pages total).

Liu et al., "Efficient MPEG-2 to MPEG-4 Video Transcoding", Proc. SPIE 5022, Image and Video Communications and Processing 2003, (May 7, 2003), pp. 186-195 (10 pages total).

Liu et al., "Global/Local Motion-Compensated Frame Interpolation for Low Bitrate Video", Apr. 2000, Proceedings of SPIE—The International Society for Optical Engineering 3974 (12 pages total).

Liu et al., "Hybrid global-local motion compensated frame interpolation for low bit rate video coding", J. Vis. Commun. Image R., vol. 14 (2003) pp. 61-79 (19 pages total).

Liu et al., "Improved Video Coding via Adaptive Selection of Generalized Motion Prediction Modes for B Frames", Picture Coding Symposium 2001, pp. 358-361 (4 pages total).

Liu et al., "Joint Temporal-Spatial Bit Allocation for Video Coding With Dependency", IEEE Transactions on Circuits and Systems for Video Technology, vol. 15, No. 1, Jan. 2005, pp. 15-26 (12 pages total).

Liu et al., "Joint Temporal-Spatial Rate Control for Adaptive Video Transcoding", 2003 International Conference on Multimedia and Expo. ICME '03. Proceedings (Cat. No. 03TH8698), Baltimore, MD, USA, 2003, pp. II-225 (4 pages total).

Liu et al., "Joint Temporal-Spatial Rate Control with Approximating Rate-Distortion Models", Proceedings of SPIE—The International Society for Optical Engineering, 2002, 4671:746-755 (10 pages total).

Liu et al., "MCI-embedded motion-compensated prediction for quality enhancement of frame interpolation", Proc. SPIE 4209, Multimedia Systems and Applications III, (Mar. 22, 2001), pp. 251-261 (12 pages total).

Liu et al., "MPEG Video Transcoding with Joint Temporal-Spatial Rate Control", Proc. SPIE 4790, Applications of Digital Image Processing XXV, (Nov. 21, 2002), pp. 278-289 (13 pages total).

Liu et al., "Nonlinear motion-compensated interpolation for low-bit-rate video", Proc. SPIE 4115, Applications of Digital Image Processing XXIII, (Dec. 28, 2000), pp. 203-213 (12 pages total).

Liu et al., "Rectangular Partitioning for Intra Prediction in HEVC", 2012 Visual Communications and Image Processing, San Diego, CA, 2012, pp. 1-6 (6 pages total).

Liu et al., "Video Prediction Block Structure and the Emerging High Efficiency Video Coding Standard", Proceedings of The 2012 Asia Pacific Signal and Information Processing Association Annual Summit and Conference, Hollywood, CA, 2012, pp. 1-4 (4 pages total).

Lou et al., "Complexity and Memory Efficient GOP Structures Supporting VCR Functionalities in H.264/AVC", 2008 IEEE International Symposium on Circuits and Systems, Seattle, WA, 2008, pp. 636-639 (4 pages total).

Lou et al., "Trick-Play Optimization for H.264 Video Decoding", Journal of Information Hiding and Multimedia Signal Processing, 2009, pp. 1-13 (15 pages total).

Pu et al., "Palette Mode Coding in HEVC Screen Content Coding Extension", IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 6, No. 4, Dec. 2016, pp. 420-432 (13 pages total).

(56)

References Cited**OTHER PUBLICATIONS**

Racape et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 11th Meeting: Ljubljana, SI, Jul. 10-18, 2018, JVET-K0500_r1-r4, (45 pages total).

Sun et al., "Improved Palette Index Map Coding On HEVC SCC", 2016 IEEE International Conference on Image Processing (ICIP), Phoenix, AZ, 2016, pp. 4210-4214 (5 pages total).

Sun et al., "Palette Mode—A New Coding Tool in Screen Content Coding Extensions of HEVC", 2015 IEEE International Conference on Image Processing (ICIP), Quebec City, QC, 2015, pp. 2409-2413 (5 pages total).

Thuong Nguyen Canh et al., "Performance Evaluation of AV1 Intra Coding Tools", 2019 International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB), IEEE, 2019, pp. 1-4 (5 pages total).

Written Opinion of the International Searching Authority dated Apr. 26, 2021 in International Application No. PCT/US2021/016203.

Xu et al., "Block Vector Prediction in Intra Block Copy for HEVC Screen Content Coding", 2015 Data Compression Conference, Snowbird, UT, 2015, pp. 273-282 (10 pages total).

Xu et al., "Intra Block Copy in HEVC Screen Content Coding Extensions", IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 6, No. 4, pp. 409-419, Dec. 2016 (11 pages total).

Xu et al., "PU Level Intra Block Copying with Flipping Mode", Signal and Information Processing Association Annual Summit and Conference (APSIPA), 2014 Asia-Pacific, Siem Reap, 2014, pp. 1-7 (7 pages total).

Yue Chen et al., "An Overview of Coding Tools in AV1: the First Video Codec from the Alliance for Open Media", SIP, 2020, vol. 9, pp. 1-15 (15 pages total).

Zhang et al., "Fast Adaptive Multiple Transform for Versatile Video Coding", 2019 Data Compression Conference, IEEE, pp. 63-72 (10 pages total).

Zhang et al., "Fast DST-7/DCT-8 with Dual Implementation Support for Versatile Video Coding", IEEE Transactions on Circuits and Systems for Video Technology, 2020 IEEE, pp. 1-17 (17 pages total).

Zhang et al., "Intra Mode Coding in HEVC Standard", 2012 Visual Communications and Image Processing, San Diego, CA, 2012, pp. 1-6 (6 pages total).

Zhang et al., "Novel Statistical Modeling, Analysis and Implementation of Rate-Distortion Estimation for H.264/AVC Coders", IEEE Transactions on Circuits and Systems for Video Technology, vol. 20, No. 5, May 2010, pp. 647-660 (14 pages total).

Zhao et al., "Coupled Primary and Secondary Transform for Next Generation Video Coding", 2018 IEEE Visual Communications and Image Processing (VCIP), Taichung, Taiwan, 2018, pp. 1-4 (4 pages total).

Zhao et al., "Joint Separable and Non-Separable Transforms for Next-Generation Video Coding", IEEE Transactions on Image Processing, vol. 27, No. 5, pp. 2514-2525, May 2018 (13 pages total).

Zhao et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 12th Meeting: Macao, CN, Oct. 3-12, 2018, JVET-L0285-r1 (18 pages total).

Zhao et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, JVET-O0545-v2 (6 pages total).

Zhao et al., "Low-Complexity Intra Prediction Refinements for Video Coding", 2018 Picture Coding Symposium (PCS), San Francisco, CA, 2018, pp. 139-143 (5 pages total).

Zhao et al., "NSST: Non-Separable Secondary Transforms for Next Generation Video Coding", 2016 Picture Coding Symposium (PCS), Nuremberg, 2016, pp. 1-5 (5 pages total).

Zhao et al., "Wide Angular Intra Prediction for Versatile Video Coding", 2019 Data Compression Conference (DCC), Snowbird, UT, USA, 2019, pp. 53-62 (10 pages total).

Zhao et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 13th Meeting: Marrakech, MA, Jan. 9-18, 2019, JVET-M0497 (11 pages total).

Zhao et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 14th Meeting: Geneva, CH, Mar. 19-27, 2019, JVET-N0394-r2 (25 pages total).

Zhao et al., Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, JVET-O0539-v2 (19 pages total).

Jianle CHEN et al., "Algorithm description for Versatile Video Coding and Test Model 5 (VTM 5)", Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, JVET-N1002-v2, Mar. 19-27, 2019, 14th Meeting, Geneva, CH (6 pages).

Office Action issued Aug. 8, 2023 in Japanese Application No. 2022-526721.

* cited by examiner

100

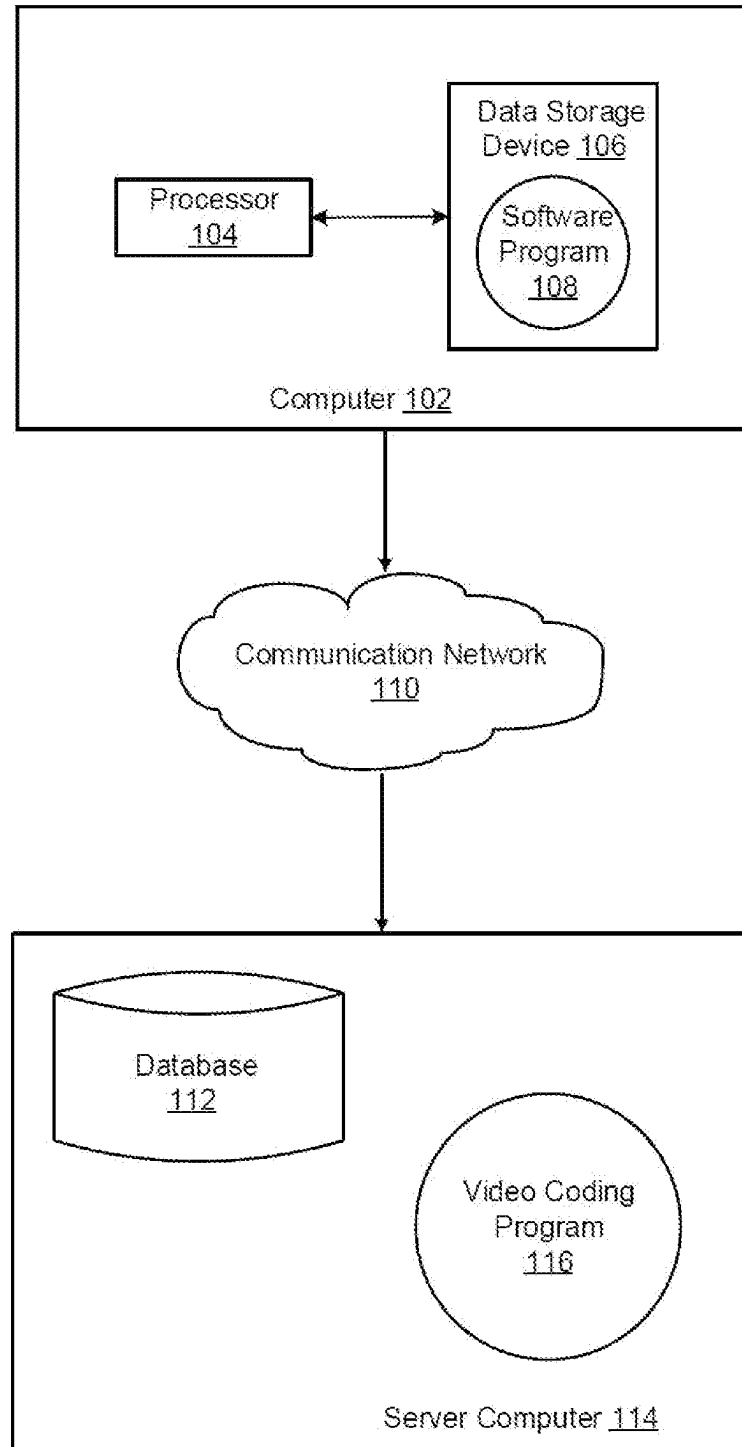


FIG. 1

200 

FIG. 2

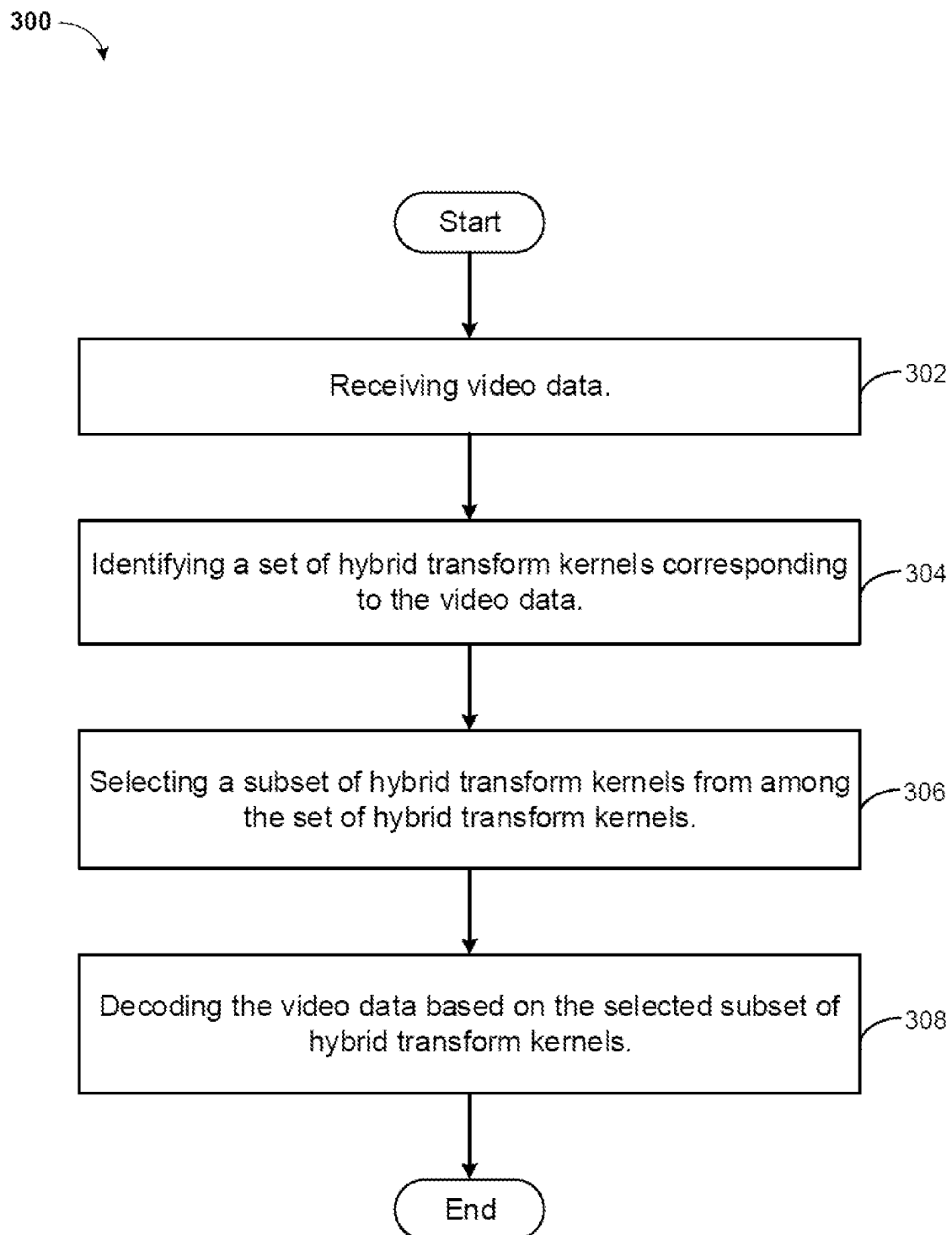


FIG. 3

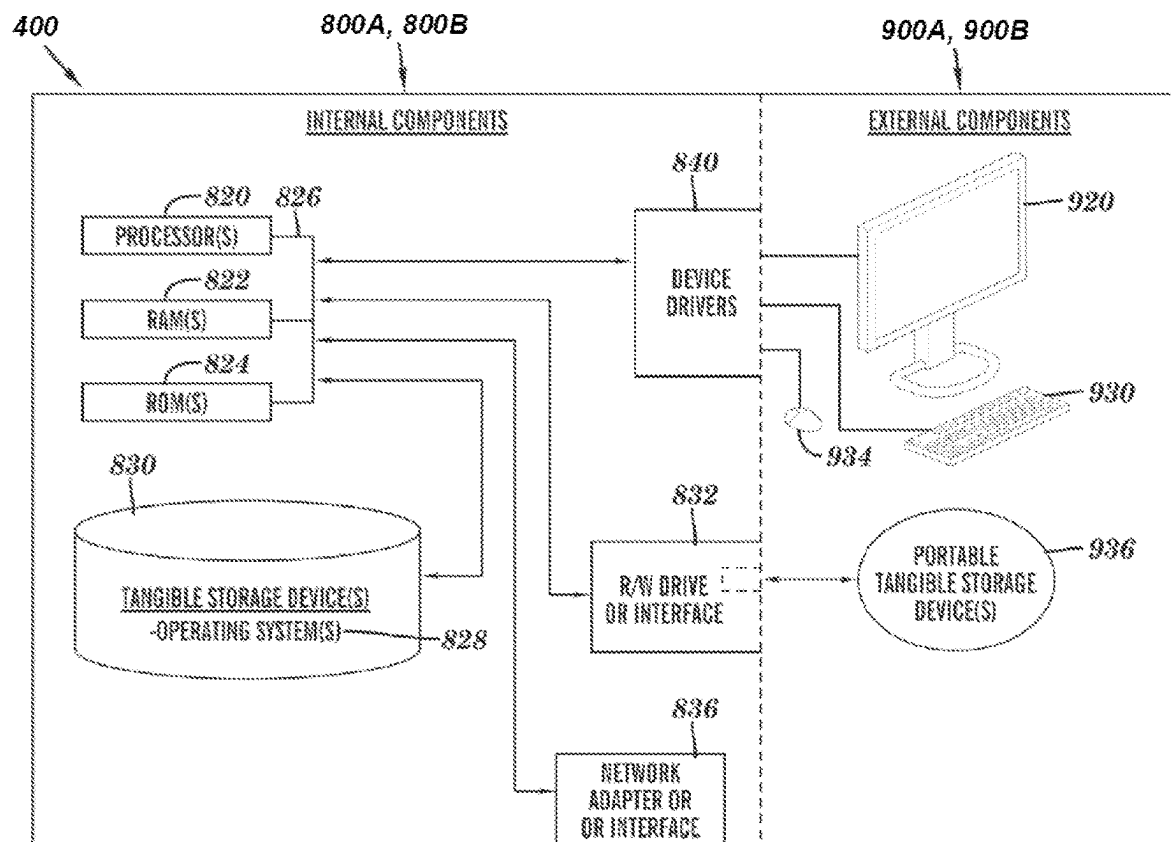


FIG. 4

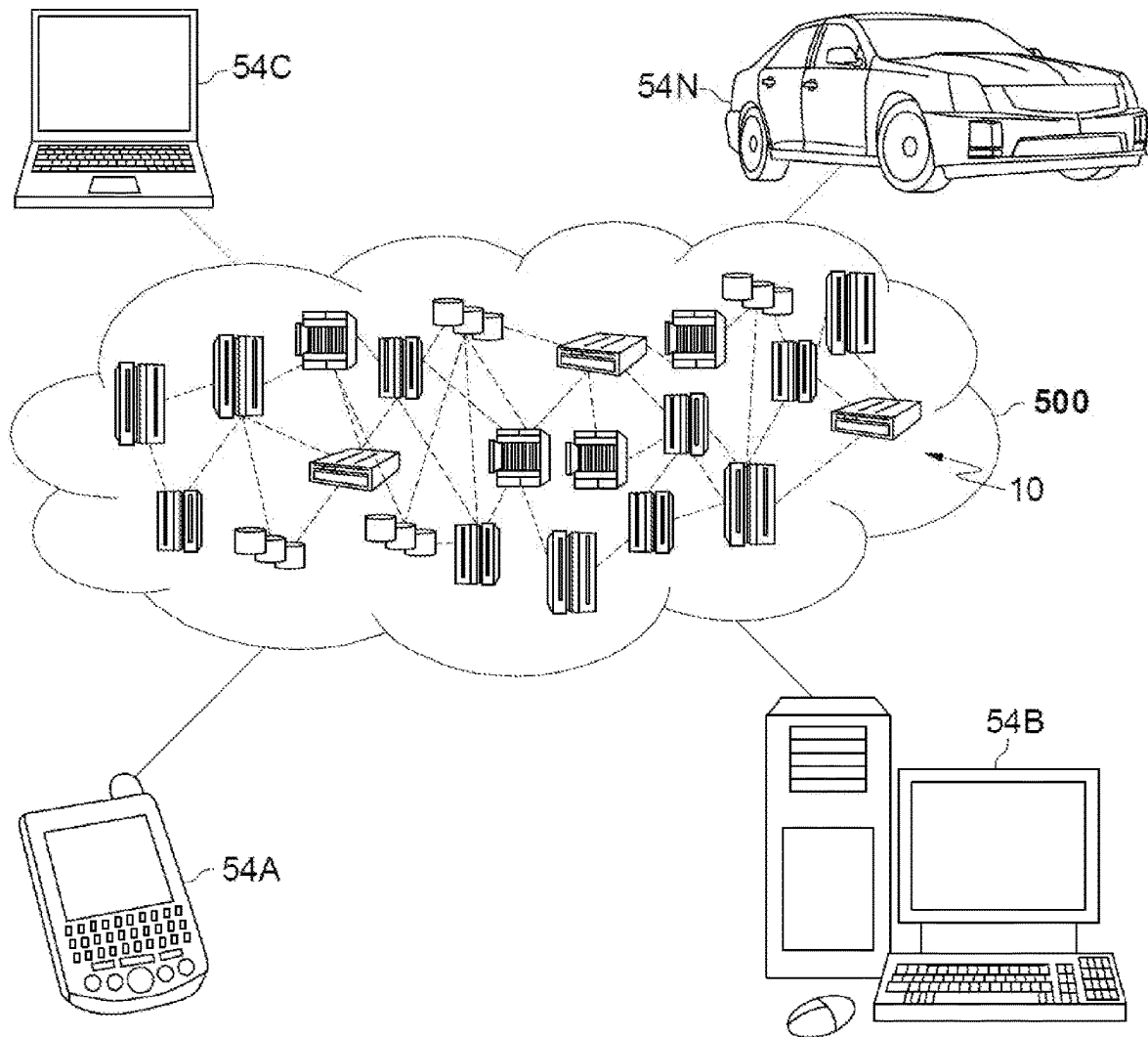


FIG. 5

600

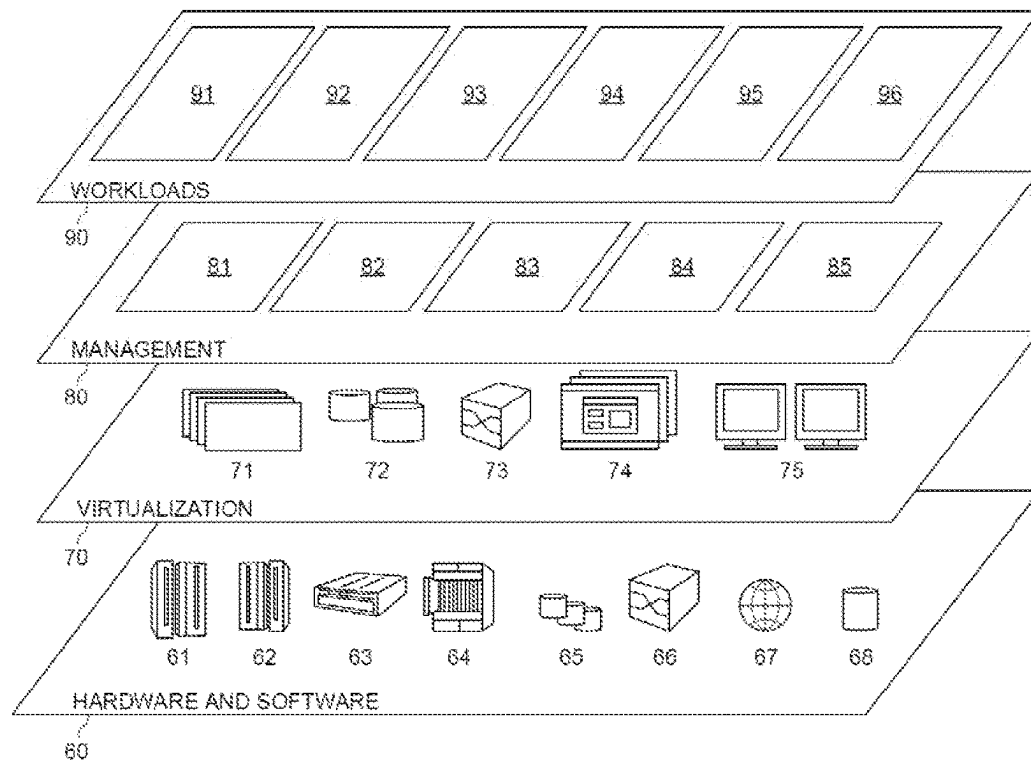


FIG. 6

1

IMPLICIT MODE DEPENDENT PRIMARY TRANSFORMS

CROSS-REFERENCE TO RELATED APPLICATION

This application is a continuation of U.S. patent application Ser. No. 17/066,791, filed on Oct. 9, 2020, in the U.S. Patent and Trademark Office, which claims the benefit of priority to U.S. Provisional Application No. 63/032,216, filed on May 29, 2020, the contents of each of which being herein incorporated by reference in their entireties.

FIELD

This disclosure relates generally to field of data processing, and more particularly to video encoding and decoding.

BACKGROUND

AOMedia Video 1 (AV1) is an open video coding format designed for video transmissions over the Internet. It was developed as a successor to VP9 by the Alliance for Open Media (AOMedia), a consortium founded in 2015 that includes semiconductor firms, video on demand providers, video content producers, software development companies and web browser vendors. Many of the components of the AV1 project were sourced from previous research efforts by Alliance members. Individual contributors started experimental technology platforms years before: Xiph's/Mozilla's Daala already published code in 2010, Google's experimental VP9 evolution project VP10 was announced on 12 Sep. 2014, and Cisco's Thor was published on 11 Aug. 2015. Building on the codebase of VP9, AV1 incorporates additional techniques, several of which were developed in these experimental formats. The first version 0.1.0 of the AV1 reference codec was published on 7 Apr. 2016. The Alliance announced the release of the AV1 bitstream specification on 28 Mar. 2018, along with a reference, software-based encoder and decoder. On 25 Jun. 2018, a validated version 1.0.0 of the specification was released. On 8 Jan. 2019 a validated version 1.0.0 with Errata 1 of the specification was released. The AV1 bitstream specification includes a reference video codec.

SUMMARY

Embodiments relate to a method, system, and computer readable medium for coding video data. According to one aspect, a method for coding video data is provided. The method may include receiving video data. A set of hybrid transform kernels corresponding to the video data is identified. A subset of hybrid transform kernels is selected, either explicitly or implicitly, from among the set of hybrid transform kernels. The video data is decoded based on the selected subset of hybrid transform kernels.

According to another aspect, a computer system for coding video data is provided. The computer system may include one or more processors, one or more computer-readable memories, one or more computer-readable tangible storage devices, and program instructions stored on at least one of the one or more storage devices for execution by at least one of the one or more processors via at least one of the one or more memories, whereby the computer system is capable of performing a method. The method may include receiving video data. A set of hybrid transform kernels corresponding to the video data is identified. A subset of

2

hybrid transform kernels is selected, either explicitly or implicitly, from among the set of hybrid transform kernels. The video data is decoded based on the selected subset of hybrid transform kernels.

According to yet another aspect, a computer readable medium for coding video data is provided. The computer readable medium may include one or more computer-readable storage devices and program instructions stored on at least one of the one or more tangible storage devices, the program instructions executable by a processor. The program instructions are executable by a processor for performing a method that may accordingly include receiving video data. A set of hybrid transform kernels corresponding to the video data is identified. A subset of hybrid transform kernels is selected, either explicitly or implicitly, from among the set of hybrid transform kernels. The video data is decoded based on the selected subset of hybrid transform kernels.

BRIEF DESCRIPTION OF THE DRAWINGS

These and other objects, features and advantages will become apparent from the following detailed description of illustrative embodiments, which is to be read in connection with the accompanying drawings. The various features of the drawings are not to scale as the illustrations are for clarity in facilitating the understanding of one skilled in the art in conjunction with the detailed description. In the drawings:

FIG. 1 illustrates a networked computer environment according to at least one embodiment;

FIG. 2 is an exemplary line graph transform, according to at least one embodiment;

FIG. 3 is an operational flowchart illustrating the steps carried out by a program that codes video data based on selecting hybrid transform kernels implicitly or explicitly, according to at least one embodiment;

FIG. 4 is a block diagram of internal and external components of computers and servers depicted in FIG. 1 according to at least one embodiment;

FIG. 5 is a block diagram of an illustrative cloud computing environment including the computer system depicted in FIG. 1, according to at least one embodiment; and

FIG. 6 is a block diagram of functional layers of the illustrative cloud computing environment of FIG. 5, according to at least one embodiment.

DETAILED DESCRIPTION

Detailed embodiments of the claimed structures and methods are disclosed herein; however, it can be understood that the disclosed embodiments are merely illustrative of the claimed structures and methods that may be embodied in various forms. Those structures and methods may, however, be embodied in many different forms and should not be construed as limited to the exemplary embodiments set forth herein. Rather, these exemplary embodiments are provided so that this disclosure will be thorough and complete and will fully convey the scope to those skilled in the art. In the description, details of well-known features and techniques may be omitted to avoid unnecessarily obscuring the presented embodiments.

Embodiments relate generally to the field of data processing, and more particularly to video encoding and decoding. The following described exemplary embodiments provide a system, method and computer program to, among other things, encode and decode video data based on selecting hybrid transform kernels, either implicitly or explicitly.

Therefore, some embodiments have the capacity to improve the field of computing by allowing for increased coding efficiency through the use of hybrid transform kernels that may be implied by the computer from the video data.

As previously described, AOMedia Video 1 (AV1) is an open video coding format designed for video transmissions over the Internet. It was developed as a successor to VP9 by the Alliance for Open Media (AOMedia), a consortium founded in 2015 that includes semiconductor firms, video on demand providers, video content producers, software development companies and web browser vendors. Many of the components of the AV1 project were sourced from previous research efforts by Alliance members. Individual contributors started experimental technology platforms years before: Xiph's/Mozilla's Daala already published code in 2010, Google's experimental VP9 evolution project VP10 was announced on 12 Sep. 2014, and Cisco's Thor was published on 11 Aug. 2015. Building on the codebase of VP9, AV1 incorporates additional techniques, several of which were developed in these experimental formats. The first version 0.1.0 of the AV1 reference codec was published on 7 Apr. 2016. The Alliance announced the release of the AV1 bitstream specification on 28 Mar. 2018, along with a reference, software-based encoder and decoder. On 25 Jun. 2018, a validated version 1.0.0 of the specification was released. On 8 Jan. 2019 a validated version 1.0.0 with Errata 1 of the specification was released. The AV1 bitstream specification includes a reference video codec.

Unlike VP9 where each coding block has only one transform type, AV1 allows each transform block to choose its own transform kernel independently. AV1 utilizes a set of hybrid transform kernels for coding the intra predicted residuals. Hybrid transform kernels generally refer to 2-D separable transform kernels extended to combinations of various 1-D kernels, such as, DCT, ADST, flipped ADST (FLIPADST), and identity transform (IDTX). The set of hybrid transform kernels and their availability for luma intra predicted residuals depend on the size of the residual block. For chroma intra predicted residuals, the transform type selection is done in an implicit way depending on the intra prediction mode. However, with the introduction of LGT's (and their flipped versions) and KLT's in the AV2 development process, the set of available hybrid transform kernels to code luma & chroma intra predicted residuals have expanded. Selecting a specific hybrid transform type from this expanded set and signaling them in the bitstream for each residual coding block incurs additional computational complexity and bitrate overhead. It may be advantageous, therefore, to use intra mode dependent and residual block size dependent LGT's & KLT's in an implicit way to utilize the directionality of variation in the magnitude of the residuals to improve the coding performance while also reducing the computational complexity and bitrate overhead.

Aspects are described herein with reference to flowchart illustrations and/or block diagrams of methods, apparatus (systems), and computer readable media according to the various embodiments. It will be understood that each block of the flowchart illustrations and/or block diagrams, and combinations of blocks in the flowchart illustrations and/or block diagrams, can be implemented by computer readable program instructions.

Referring now to FIG. 1, a functional block diagram of a networked computer environment illustrating a video coding system 100 (hereinafter "system") for encoding and decoding video data based on selecting hybrid transform kernels implicitly or explicitly. It should be appreciated that FIG. 1

provides only an illustration of one implementation and does not imply any limitations with regard to the environments in which different embodiments may be implemented. Many modifications to the depicted environments may be made based on design and implementation requirements.

The system 100 may include a computer 102 and a server computer 114. The computer 102 may communicate with the server computer 114 via a communication network 110 (hereinafter "network"). The computer 102 may include a processor 104 and a software program 108 that is stored on a data storage device 106 and is enabled to interface with a user and communicate with the server computer 114. As will be discussed below with reference to FIG. 4 the computer 102 may include internal components 800A and external components 900A, respectively, and the server computer 114 may include internal components 800B and external components 900B, respectively. The computer 102 may be, for example, a mobile device, a telephone, a personal digital assistant, a netbook, a laptop computer, a tablet computer, a desktop computer, or any type of computing devices capable of running a program, accessing a network, and accessing a database.

The server computer 114 may also operate in a cloud computing service model, such as Software as a Service (SaaS), Platform as a Service (PaaS), or Infrastructure as a Service (IaaS), as discussed below with respect to FIGS. 5 and 6. The server computer 114 may also be located in a cloud computing deployment model, such as a private cloud, community cloud, public cloud, or hybrid cloud.

The server computer 114, which may be used for coding video data based on selecting hybrid transform kernels implicitly or explicitly is enabled to run a Video Coding Program 116 (hereinafter "program") that may interact with a database 112. The Video Coding Program method is explained in more detail below with respect to FIG. 3. In one embodiment, the computer 102 may operate as an input device including a user interface while the program 116 may run primarily on server computer 114. In an alternative embodiment, the program 116 may run primarily on one or more computers 102 while the server computer 114 may be used for processing and storage of data used by the program 116. It should be noted that the program 116 may be a standalone program or may be integrated into a larger video coding program.

It should be noted, however, that processing for the program 116 may, in some instances be shared amongst the computers 102 and the server computers 114 in any ratio. In another embodiment, the program 116 may operate on more than one computer, server computer, or some combination of computers and server computers, for example, a plurality of computers 102 communicating across the network 110 with a single server computer 114. In another embodiment, for example, the program 116 may operate on a plurality of server computers 114 communicating across the network 110 with a plurality of client computers. Alternatively, the program may operate on a network server communicating across the network with a server and a plurality of client computers.

The network 110 may include wired connections, wireless connections, fiber optic connections, or some combination thereof. In general, the network 110 can be any combination of connections and protocols that will support communications between the computer 102 and the server computer 114. The network 110 may include various types of networks, such as, for example, a local area network (LAN), a wide area network (WAN) such as the Internet, a telecommunication network such as the Public Switched Telephone

5

Network (PSTN), a wireless network, a public switched network, a satellite network, a cellular network (e.g., a fifth generation (5G) network, a long-term evolution (LTE) network, a third generation (3G) network, a code division multiple access (CDMA) network, etc.), a public land mobile network (PLMN), a metropolitan area network (MAN), a private network, an ad hoc network, an intranet, a fiber optic-based network, or the like, and/or a combination of these or other types of networks.

The number and arrangement of devices and networks shown in FIG. 1 are provided as an example. In practice, there may be additional devices and/or networks, fewer devices and/or networks, different devices and/or networks, or differently arranged devices and/or networks than those shown in FIG. 1. Furthermore, two or more devices shown in FIG. 1 may be implemented within a single device, or a single device shown in FIG. 1 may be implemented as multiple, distributed devices. Additionally, or alternatively, a set of devices (e.g., one or more devices) of system 100 may perform one or more functions described as being performed by another set of devices of system 100.

Referring now to FIG. 2, an exemplary line graph transform (LGT) 200 is depicted. Graphs may be generic mathematical structures consisting of sets of vertices and edges, which are used for modelling affinity relations between the objects of interest. In practice, weighted graphs (for which a set of weights are assigned to edges and potentially to vertices) may provide sparse representations for robust modeling of signals/data. LGTs can improve coding efficiency by providing a better adaptation for diverse block statistics. Separable LGTs may be designed and optimized by learning line graphs from data to model underlying row and column-wise statistics of blocks residual signals, where the associated generalized graph Laplacian (GGL) matrices are used to derive LGTs.

For example, given a weighted graph $G(W, V)$ a GGL matrix may be defined as $L_c = D - W + V$, where W may be the adjacency matrix consisting of non-negative edge weights w_c , D may be the diagonal degree matrix, and V may be the diagonal matrix denoting weighted self-loops v_{c1} , v_{c2} . The matrix L_c can be represented as:

$$L_c = \begin{bmatrix} w_c + v_{c1} & -w_c & & 0 \\ -w_c & 2w_c & -w_c & \\ & \ddots & \ddots & \ddots \\ & & -w_c & 2w_c & -w_c \\ 0 & & & -w_c & w_c + v_{c2} \end{bmatrix} \quad \text{for } w_c > 0. \quad (\text{Eq. 1})$$

The LGTs can then be derived by the eigen-decomposition of the GGL L_c : $L_c = U\Phi U^T$, where columns of orthogonal matrix U are the basis vectors of the LGT, and Φ is the diagonal eigenvalue matrix. In fact, discrete cosine transforms (DCTs) and discrete sine transforms (DSTs), including DCT-2, DCT-8 and DST-7, are LGTs derived from certain forms of GGLs. DCT-2 is derived by setting $v_{c1}=0$. DST-7 is derived by setting $v_{c1}=w_c$. DCT-8 is derived by setting $v_{c1}=w_c$. DST-4 is derived by setting $v_{c1}=2w_c$. DCT-4 is derived by setting $v_{c2}=2w_c$.

The LGTs are implemented using matrix multiplications for transform sizes 4, 8 & 16. The 4-point LGT core is derived by setting $v_{c1}=2w_c$ in L_c , which means that it is a DST-4. The 8-point LGT core is derived by setting $v_{c2}=1.5w_c$ in L_c & the 16-point LGT core is derived by setting $v_{c1}=w_c$ in L_c , which means that it is a DST-7.

The extended set of hybrid transform kernels may be referred to as set A. Set A exhaustively includes all combi-

6

nations of discrete cosine transform (DCT), identity transform (IDTX, which skips transform coding in a certain direction), asymmetric discrete sine transform (ADST), flipped asymmetric discrete sine transform (FLIPADST, which applies ADST in reverse order), line graph transform (LGT), flipped line graph transform (FLIPLGT), Karhunen-Loeve Transform (KLT), etc. A subset of elements of A, which may be a reduced set of transform types, may be referred to as x , such that $x \in A$. The subset x may include one or more transform types (e.g., DCT, ADST, LGT, KLT) and/or one or more combinations of vertical and horizontal transform types (e.g., DCT_DCT, LGT_LGT, DCT_LGT, LGT_DCT).

According to one or more embodiments, an implicit method may be used to select an element of x , such that the hybrid transform type may be selected based on coded information that is available to both encoder and decoder. Therefore, no additional signaling may be needed to specify the transform type at decoder. In one embodiment, the selection may be made depending on the intra prediction mode and/or the block size. In one embodiment, one or more among the 8 nominal modes, 5 non-angular smooth modes and the angle delta value (e.g., $-3 \sim +3$) may be considered during the selection process. In one embodiment, for directional intra prediction modes, only the nominal mode may be used to select the transform type (i.e., directional intra prediction modes which share the same nominal modes but different angle delta values may apply the same implicit transform type).

In one embodiment, for recursive-filtering modes and DC mode select the same hybrid transform type. In one embodiment, recursive-filtering modes and SMOOTH mode select the same hybrid transform type. In one embodiment, SMOOTH, SMOOTH_H, SMOOTH_V modes select the same hybrid transform type. In one embodiment, SMOOTH, SMOOTH_H, SMOOTH_V and Paeth prediction modes select the same hybrid transform type. In one embodiment, recursive-filtering modes, SMOOTH, and Paeth prediction modes select the same hybrid transform type. In one embodiment, Vertical mode (V_PRED) and SMOOTH_V prediction modes select the same hybrid transform type. In one embodiment, Horizontal mode (H_PRED) and SMOOTH_H prediction modes select the same hybrid transform type. In one embodiment, chroma-from-luma (CfL) mode and DC mode select the same hybrid transform type. In one embodiment, CfL mode and SMOOTH mode select the same hybrid transform type. In one embodiment, CfL mode and Paeth mode select the same hybrid transform type.

In one embodiment, depending on one or more among the 8 nominal modes, 5 non-angular smooth modes, the angle delta value (e.g., $-3 \sim +3$) and block size, LGT with different self-loop weights (v_{c1} , v_{c2}) may be used. In one embodiment, depending on one or more among the 8 nominal modes, 5 non-angular smooth, the angle delta value (e.g., $-3 \sim +3$) and block size, KLT's with different statistical properties may be used. In one embodiment, for the same intra prediction mode that may be enabled for both luma and chroma components, the implicit hybrid transform selection may be same.

According to one or more embodiments, an explicit method may be proposed to select an element of x , such that the selection may need to be identified by a syntax signaled in the bitstreams (i.e., the encoder needs to explicitly chose and signal the transform type at block-level). The block-level may include superblock level, coding block level, prediction block level or transform block level. In one embodiment, explicit transform scheme (at least two trans-

form type candidates) can be applied for all intra prediction mode, but the number of hybrid transform candidates can be different for different intra prediction modes. In another embodiment, for some intra prediction mode, either an implicit or explicit transform scheme can be used, while other intra prediction modes apply the implicit transform scheme (only one transform type available). In one embodiment, when the explicit transform scheme involves the use of LGT, identifiers of the self-loop weights (v_{c1} , v_{c2}) that specify the LGT candidate can be signaled in the bitstream at block-level, the identifier can be either an index of the associated self-loop rate value or the self-loop rate values. In one embodiment, when the explicit transform scheme involves the use of KLT, identifiers of the KLT kernel can be signaled in the bitstream at block-level, the identifier can be either an index of the KLT or the KLT matrix element values.

A switch between an explicit method and an implicit method can be indicated at either high-level syntax or at block-level. When the selection may be indicated at HLS, it may include, video parameter set (VPS), sequence parameter set (SPS), picture parameter set (PPS), slice header. When the switch may be indicated at block level, it may include superblock level, coding block level, prediction block level, and/or transform block level.

Referring now to FIG. 3, an operational flowchart illustrating the steps of a method 300 for coding video data is depicted. In some implementations, one or more process blocks of FIG. 3 may be performed by the computer 102 (FIG. 1) and the server computer 114 (FIG. 1). In some implementations, one or more process blocks of FIG. 3 may be performed by another device or a group of devices separate from or including the computer 102 and the server computer 114.

At 302, the method 300 includes receiving video data.

At 304, the method 300 includes identifying a set of hybrid transform kernels corresponding to the video data.

At 306, the method 300 includes selecting a subset of hybrid transform kernels, either explicitly or implicitly, from among the set of hybrid transform kernels.

At 308, the method 300 includes decoding the video data based on the selected subset of hybrid transform kernels.

It may be appreciated that FIG. 3 provides only an illustration of one implementation and does not imply any limitations with regard to how different embodiments may be implemented. Many modifications to the depicted environments may be made based on design and implementation requirements.

FIG. 4 is a block diagram 400 of internal and external components of computers depicted in FIG. 1 in accordance with an illustrative embodiment. It should be appreciated that FIG. 4 provides only an illustration of one implementation and does not imply any limitations with regard to the environments in which different embodiments may be implemented. Many modifications to the depicted environments may be made based on design and implementation requirements.

Computer 102 (FIG. 1) and server computer 114 (FIG. 1) may include respective sets of internal components 800A,B and external components 900A,B illustrated in FIG. 4. Each of the sets of internal components 800 include one or more processors 820, one or more computer-readable RAMs 822 and one or more computer-readable ROMs 824 on one or more buses 826, one or more operating systems 828, and one or more computer-readable tangible storage devices 830.

Processor 820 is implemented in hardware, firmware, or a combination of hardware and software. Processor 820 is a central processing unit (CPU), a graphics processing unit

(GPU), an accelerated processing unit (APU), a microprocessor, a microcontroller, a digital signal processor (DSP), a field-programmable gate array (FPGA), an application-specific integrated circuit (ASIC), or another type of processing component. In some implementations, processor 820 includes one or more processors capable of being programmed to perform a function. Bus 826 includes a component that permits communication among the internal components 800A,B.

The one or more operating systems 828, the software program 108 (FIG. 1) and the Video Coding Program 116 (FIG. 1) on server computer 114 (FIG. 1) are stored on one or more of the respective computer-readable tangible storage devices 830 for execution by one or more of the respective processors 820 via one or more of the respective RAMs 822 (which typically include cache memory). In the embodiment illustrated in FIG. 4, each of the computer-readable tangible storage devices 830 is a magnetic disk storage device of an internal hard drive. Alternatively, each of the computer-readable tangible storage devices 830 is a semiconductor storage device such as ROM 824, EPROM, flash memory, an optical disk, a magneto-optic disk, a solid state disk, a compact disc (CD), a digital versatile disc (DVD), a floppy disk, a cartridge, a magnetic tape, and/or another type of non-transitory computer-readable tangible storage device that can store a computer program and digital information.

Each set of internal components 800A,B also includes a R/W drive or interface 832 to read from and write to one or more portable computer-readable tangible storage devices 936 such as a CD-ROM, DVD, memory stick, magnetic tape, magnetic disk, optical disk or semiconductor storage device. A software program, such as the software program 108 (FIG. 1) and the Video Coding Program 116 (FIG. 1) can be stored on one or more of the respective portable computer-readable tangible storage devices 936, read via the respective R/W drive or interface 832 and loaded into the respective hard drive 830.

Each set of internal components 800A,B also includes network adapters or interfaces 836 such as a TCP/IP adapter cards; wireless Wi-Fi interface cards; or 3G, 4G, or 5G wireless interface cards or other wired or wireless communication links. The software program 108 (FIG. 1) and the Video Coding Program 116 (FIG. 1) on the server computer 114 (FIG. 1) can be downloaded to the computer 102 (FIG. 1) and server computer 114 from an external computer via a network (for example, the Internet, a local area network or other, wide area network) and respective network adapters or interfaces 836. From the network adapters or interfaces 836, the software program 108 and the Video Coding Program 116 on the server computer 114 are loaded into the respective hard drive 830. The network may comprise copper wires, optical fibers, wireless transmission, routers, firewalls, switches, gateway computers and/or edge servers.

Each of the sets of external components 900A,B can include a computer display monitor 920, a keyboard 930, and a computer mouse 934. External components 900A,B can also include touch screens, virtual keyboards, touch pads, pointing devices, and other human interface devices. Each of the sets of internal components 800A,B also includes device drivers 840 to interface to computer display monitor 920, keyboard 930 and computer mouse 934. The device drivers 840, R/W drive or interface 832 and network adapter or interface 836 comprise hardware and software (stored in storage device 830 and/or ROM 824).

It is understood in advance that although this disclosure includes a detailed description on cloud computing, implementation of the teachings recited herein are not limited to

a cloud computing environment. Rather, some embodiments are capable of being implemented in conjunction with any other type of computing environment now known or later developed.

Cloud computing is a model of service delivery for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g. networks, network bandwidth, servers, processing, memory, storage, applications, virtual machines, and services) that can be rapidly provisioned and released with minimal management effort or interaction with a provider of the service. This cloud model may include at least five characteristics, at least three service models, and at least four deployment models.

Characteristics are as follows:

On-demand self-service: a cloud consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with the service's provider.

Broad network access: capabilities are available over a network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and PDAs).

Resource pooling: the provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to demand. There is a sense of location independence in that the consumer generally has no control or knowledge over the exact location of the provided resources but may be able to specify location at a higher level of abstraction (e.g., country, state, or datacenter).

Rapid elasticity: capabilities can be rapidly and elastically provisioned, in some cases automatically, to quickly scale out and rapidly released to quickly scale in. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time.

Measured service: cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported providing transparency for both the provider and consumer of the utilized service.

Service Models are as follows:

Software as a Service (SaaS): the capability provided to the consumer is to use the provider's applications running on a cloud infrastructure. The applications are accessible from various client devices through a thin client interface such as a web browser (e.g., web-based e-mail). The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, storage, or even individual application capabilities, with the possible exception of limited user-specific application configuration settings.

Platform as a Service (PaaS): the capability provided to the consumer is to deploy onto the cloud infrastructure consumer-created or acquired applications created using programming languages and tools supported by the provider. The consumer does not manage or control the underlying cloud infrastructure including networks, servers, operating systems, or storage, but has control over the deployed applications and possibly application hosting environment configurations.

Infrastructure as a Service (IaaS): the capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which can include operating systems and applications. The consumer does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, deployed applications, and possibly limited control of select networking components (e.g., host firewalls).

Deployment Models are as follows:

Private cloud: the cloud infrastructure is operated solely for an organization. It may be managed by the organization or a third party and may exist on-premises or off-premises.

Community cloud: the cloud infrastructure is shared by several organizations and supports a specific community that has shared concerns (e.g., mission, security requirements, policy, and compliance considerations). It may be managed by the organizations or a third party and may exist on-premises or off-premises.

Public cloud: the cloud infrastructure is made available to the general public or a large industry group and is owned by an organization selling cloud services.

Hybrid cloud: the cloud infrastructure is a composition of two or more clouds (private, community, or public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load-balancing between clouds).

A cloud computing environment is service oriented with a focus on statelessness, low coupling, modularity, and semantic interoperability. At the heart of cloud computing is an infrastructure comprising a network of interconnected nodes.

Referring to FIG. 5, illustrative cloud computing environment 500 is depicted. As shown, cloud computing environment 500 comprises one or more cloud computing nodes 10 with which local computing devices used by cloud consumers, such as, for example, personal digital assistant (PDA) or cellular telephone 54A, desktop computer 54B, laptop computer 54C, and/or automobile computer system 54N may communicate. Cloud computing nodes 10 may communicate with one another. They may be grouped (not shown) physically or virtually, in one or more networks, such as Private, Community, Public, or Hybrid clouds as described hereinabove, or a combination thereof. This allows cloud computing environment 500 to offer infrastructure, platforms and/or software as services for which a cloud consumer does not need to maintain resources on a local computing device. It is understood that the types of computing devices 54A-N shown in FIG. 4 are intended to be illustrative only and that cloud computing nodes 10 and cloud computing environment 500 can communicate with any type of computerized device over any type of network and/or network addressable connection (e.g., using a web browser).

Referring to FIG. 6, a set of functional abstraction layers 600 provided by cloud computing environment 500 (FIG. 5) is shown. It should be understood in advance that the components, layers, and functions shown in FIG. 6 are intended to be illustrative only and embodiments are not limited thereto. As depicted, the following layers and corresponding functions are provided:

Hardware and software layer 60 includes hardware and software components. Examples of hardware components include: mainframes 61; RISC (Reduced Instruction Set

11

Computer) architecture based servers **62**; servers **63**; blade servers **64**; storage devices **65**; and networks and networking components **66**. In some embodiments, software components include network application server software **67** and database software **68**.

Virtualization layer **70** provides an abstraction layer from which the following examples of virtual entities may be provided: virtual servers **71**; virtual storage **72**; virtual networks **73**, including virtual private networks; virtual applications and operating systems **74**; and virtual clients **75**.

In one example, management layer **80** may provide the functions described below. Resource provisioning **81** provides dynamic procurement of computing resources and other resources that are utilized to perform tasks within the cloud computing environment. Metering and Pricing **82** provide cost tracking as resources are utilized within the cloud computing environment, and billing or invoicing for consumption of these resources. In one example, these resources may comprise application software licenses. Security provides identity verification for cloud consumers and tasks, as well as protection for data and other resources. User portal **83** provides access to the cloud computing environment for consumers and system administrators. Service level management **84** provides cloud computing resource allocation and management such that required service levels are met. Service Level Agreement (SLA) planning and fulfillment **85** provide pre-arrangement for, and procurement of, cloud computing resources for which a future requirement is anticipated in accordance with an SLA.

Workloads layer **90** provides examples of functionality for which the cloud computing environment may be utilized. Examples of workloads and functions which may be provided from this layer include: mapping and navigation **91**; software development and lifecycle management **92**; virtual classroom education delivery **93**; data analytics processing **94**; transaction processing **95**; and Video Coding **96**. Video Coding **96** may encode and decode video data based on selecting hybrid transform kernels implicitly or explicitly.

Some embodiments may relate to a system, a method, and/or a computer readable medium at any possible technical detail level of integration. The computer readable medium may include a computer-readable non-transitory storage medium (or media) having computer readable program instructions thereon for causing a processor to carry out operations.

The computer readable storage medium can be a tangible device that can retain and store instructions for use by an instruction execution device. The computer readable storage medium may be, for example, but is not limited to, an electronic storage device, a magnetic storage device, an optical storage device, an electromagnetic storage device, a semiconductor storage device, or any suitable combination of the foregoing. A non-exhaustive list of more specific examples of the computer readable storage medium includes the following: a portable computer diskette, a hard disk, a random access memory (RAM), a read-only memory (ROM), an erasable programmable read-only memory (EPROM or Flash memory), a static random access memory (SRAM), a portable compact disc read-only memory (CD-ROM), a digital versatile disk (DVD), a memory stick, a floppy disk, a mechanically encoded device such as punch-cards or raised structures in a groove having instructions recorded thereon, and any suitable combination of the foregoing. A computer readable storage medium, as used herein, is not to be construed as being transitory signals per se, such as radio waves or other freely propagating electromagnetic

12

waves, electromagnetic waves propagating through a waveguide or other transmission media (e.g., light pulses passing through a fiber-optic cable), or electrical signals transmitted through a wire.

Computer readable program instructions described herein can be downloaded to respective computing/processing devices from a computer readable storage medium or to an external computer or external storage device via a network, for example, the Internet, a local area network, a wide area network and/or a wireless network. The network may comprise copper transmission cables, optical transmission fibers, wireless transmission, routers, firewalls, switches, gateway computers and/or edge servers. A network adapter card or network interface in each computing/processing device receives computer readable program instructions from the network and forwards the computer readable program instructions for storage in a computer readable storage medium within the respective computing/processing device.

Computer readable program code/instructions for carrying out operations may be assembler instructions, instruction-set-architecture (ISA) instructions, machine instructions, machine dependent instructions, microcode, firmware instructions, state-setting data, configuration data for integrated circuitry, or either source code or object code written in any combination of one or more programming languages, including an object oriented programming language such as Smalltalk, C++, or the like, and procedural programming languages, such as the “C” programming language or similar programming languages. The computer readable program instructions may execute entirely on the user’s computer, partly on the user’s computer, as a stand-alone software package, partly on the user’s computer and partly on a remote computer or entirely on the remote computer or server. In the latter scenario, the remote computer may be connected to the user’s computer through any type of network, including a local area network (LAN) or a wide area network (WAN), or the connection may be made to an external computer (for example, through the Internet using an Internet Service Provider). In some embodiments, electronic circuitry including, for example, programmable logic circuitry, field-programmable gate arrays (FPGA), or programmable logic arrays (PLA) may execute the computer readable program instructions by utilizing state information of the computer readable program instructions to personalize the electronic circuitry, in order to perform aspects or operations.

These computer readable program instructions may be provided to a processor of a general purpose computer, special purpose computer, or other programmable data processing apparatus to produce a machine, such that the instructions, which execute via the processor of the computer or other programmable data processing apparatus, create means for implementing the functions/acts specified in the flowchart and/or block diagram block or blocks. These computer readable program instructions may also be stored in a computer readable storage medium that can direct a computer, a programmable data processing apparatus, and/or other devices to function in a particular manner, such that the computer readable storage medium having instructions stored therein comprises an article of manufacture including instructions which implement aspects of the function/act specified in the flowchart and/or block diagram block or blocks.

The computer readable program instructions may also be loaded onto a computer, other programmable data processing apparatus, or other device to cause a series of operational steps to be performed on the computer, other programmable

13

apparatus or other device to produce a computer implemented process, such that the instructions which execute on the computer, other programmable apparatus, or other device implement the functions/acts specified in the flowchart and/or block diagram block or blocks.

The flowchart and block diagrams in the Figures illustrate the architecture, functionality, and operation of possible implementations of systems, methods, and computer readable media according to various embodiments. In this regard, each block in the flowchart or block diagrams may represent a module, segment, or portion of instructions, which comprises one or more executable instructions for implementing the specified logical function(s). The method, computer system, and computer readable medium may include additional blocks, fewer blocks, different blocks, or differently arranged blocks than those depicted in the Figures. In some alternative implementations, the functions noted in the blocks may occur out of the order noted in the Figures. For example, two blocks shown in succession may, in fact, be executed concurrently or substantially concurrently, or the blocks may sometimes be executed in the reverse order, depending upon the functionality involved. It will also be noted that each block of the block diagrams and/or flowchart illustration, and combinations of blocks in the block diagrams and/or flowchart illustration, can be implemented by special purpose hardware-based systems that perform the specified functions or acts or carry out combinations of special purpose hardware and computer instructions.

It will be apparent that systems and/or methods, described herein, may be implemented in different forms of hardware, firmware, or a combination of hardware and software. The actual specialized control hardware or software code used to implement these systems and/or methods is not limiting of the implementations. Thus, the operation and behavior of the systems and/or methods were described herein without reference to specific software code—it being understood that software and hardware may be designed to implement the systems and/or methods based on the description herein.

No element, act, or instruction used herein should be construed as critical or essential unless explicitly described as such. Also, as used herein, the articles “a” and “an” are intended to include one or more items, and may be used interchangeably with “one or more.” Furthermore, as used herein, the term “set” is intended to include one or more items (e.g., related items, unrelated items, a combination of related and unrelated items, etc.), and may be used interchangeably with “one or more.” Where only one item is intended, the term “one” or similar language is used. Also, as used herein, the terms “has,” “have,” “having,” or the like are intended to be open-ended terms. Further, the phrase “based on” is intended to mean “based, at least in part, on” unless explicitly stated otherwise.

The descriptions of the various aspects and embodiments have been presented for purposes of illustration, but are not intended to be exhaustive or limited to the embodiments disclosed. Even though combinations of features are recited in the claims and/or disclosed in the specification, these combinations are not intended to limit the disclosure of possible implementations. In fact, many of these features may be combined in ways not specifically recited in the claims and/or disclosed in the specification. Although each dependent claim listed below may directly depend on only one claim, the disclosure of possible implementations includes each dependent claim in combination with every other claim in the claim set. Many modifications and variations will be apparent to those of ordinary skill in the art

14

without departing from the scope of the described embodiments. The terminology used herein was chosen to best explain the principles of the embodiments, the practical application or technical improvement over technologies found in the marketplace, or to enable others of ordinary skill in the art to understand the embodiments disclosed herein.

What is claimed is:

1. A method for video decoding, executable by a processor of a decoder, the method comprising:

receiving video data;

identifying an extended set of hybrid transform kernels as set A, wherein set A exhaustively includes all combinations of DCT, IDTX, ADST, FLIPADST, LGT, FLIPLGT, KLT, and a subset of elements of A, which is a reduced set of transform types, comprises subset x such that $x \in A$, and the subset x includes any of one or more transform types, {DCT, ADST, LGT, KLT}, and one or more combinations of vertical and horizontal transform types {DCT_DCT, LGT_LGT, DCT_LGT, LGT_DCT};

selecting an element of the subset x based on coded information, that is available to the decoder and separate from the video data, depending on any of an intra prediction mode and a block size, and by using, for directional intra prediction modes, only a nominal mode to select the element of the subset x, wherein the directional intra prediction modes share same nominal modes, including the nominal mode, and respectively comprise different angle delta values applied to a same implicit transform type; and

decoding the video data based on element of the subset x.

2. The method of claim 1, further comprising considering one or more among 8 nominal modes, 5 non-angular smooth modes and an angle delta value during selection of the element of the subset x.

3. The method of claim 1, wherein selecting of the element of the subset x comprises any of:

selecting a same hybrid transform type for recursive-filtering modes and the DC mode,

selecting a same hybrid transform type for the recursive-filtering modes and the SMOOTH mode,

selecting a same hybrid transform type for the SMOOTH mode, SMOOTH_H mode, and SMOOTH_V mode,

selecting a same hybrid transform type for the SMOOTH mode, SMOOTH_H mode, SMOOTH_V mode, and Paeth mode,

selecting a same hybrid transform type for the recursive-filtering modes, the SMOOTH mode, and Paeth mode,

selecting a same hybrid transform type for the V_PRED mode and SMOOTH_V mode,

selecting a same hybrid transform type for the H_PRED mode and the SMOOTH_H mode,

selecting a same hybrid transform type for the chroma-from-luma mode and the DC mode, or

selecting a same hybrid transform type for the chroma-from-luma mode and the SMOOTH mode.

4. The method of claim 1, wherein selecting the element of the subset x comprises any of:

selecting a same hybrid transform type for a chroma-from-luma mode and a Paeth mode, and

using LGT, with different self-loop weights (v_{c1} , v_{c2}), depending on one or more among 8 nominal modes, 5 non-angular smooth modes, an angle delta value, a block size.

5. The method of claim 1, wherein selecting the element of the subset x comprises any of:

15

using KLTs, with different statistical properties, depending on one or more among 8 nominal modes, 5 non-angular smooth modes, an angle delta value, a block size; and

selecting a same hybrid transform type for a same intra prediction mode that is enabled for both luma and chroma components.

6. A method for video encoding, executable by a processor of an encoder, the method comprising:

receiving video data;

identifying an extended set of hybrid transform kernels as set A, wherein set A exhaustively includes all combinations of DCT, IDTX, ADST, FLIPADST, LGT, FLIPLGT, KLT, and a subset of elements of A, which is a reduced set of transform types, comprises subset x such that $x \in A$, and the subset x includes any of one or more transform types, {DCT, ADST, LGT, KLT}, and one or more combinations of vertical and horizontal transform types {DCT_DCT, LGT_LGT, DCT_LGT, LGT_DCT};

selecting an element of the subset x based on information that is available to the encoder and separate from the video data, depending on any of an intra prediction mode and a block size, and by using, for directional intra prediction modes, only a nominal mode to select the element of the subset x, wherein the directional intra prediction modes share same nominal modes, including the nominal mode, and respectively comprise different angle delta values applied to a same implicit transform type; and

encoding the video data based on element of the subset x.

7. The method of claim 6, further comprising considering one or more among 8 nominal modes, 5 non-angular smooth modes and an angle delta value during selection of the element of the subset x.

8. The method of claim 6, wherein selecting of the element of the subset x comprises any of:

selecting a same hybrid transform type for recursive-filtering modes and the DC mode,

selecting a same hybrid transform type for the recursive-filtering modes and the SMOOTH mode,

selecting a same hybrid transform type for the SMOOTH mode, SMOOTH_H mode, and SMOOTH_V mode,

selecting a same hybrid transform type for the SMOOTH mode, SMOOTH_H mode, SMOOTH_V mode, and Paeth mode,

selecting a same hybrid transform type for the recursive-filtering modes, the SMOOTH mode, and Paeth mode,

selecting a same hybrid transform type for the V_PRED mode and SMOOTH_V mode, selecting a same hybrid transform type for the H_PRED mode and the SMOOTH_H mode,

selecting a same hybrid transform type for the chroma-from-luma mode and the DC mode, or

selecting a same hybrid transform type for the chroma-from-luma mode and the SMOOTH mode.

9. The method of claim 6, wherein selecting the element of the subset x comprises any of:

selecting a same hybrid transform type for a chroma-from-luma mode and a Paeth mode, and

using LGT, with different self-loop weights (v_{c1} , v_{c2}), depending on one or more among 8 nominal modes, 5 non-angular smooth modes, an angle delta value, a block size.

10. The method of claim 6, wherein selecting the element of the subset x comprises any of:

16

using KLTs, with different statistical properties, depending on one or more among 8 nominal modes, 5 non-angular smooth modes, an angle delta value, a block size; and

selecting a same hybrid transform type for a same intra prediction mode that is enabled for both luma and chroma components.

11. A method of processing visual media data, the method comprising:

performing a conversion between a visual media file and a bitstream of a visual media data according to a format rule, the format rule indicating to identify an extended set of hybrid transform kernels as set A, wherein set A exhaustively includes all combinations of DCT, IDTX, ADST, FLIPADST, LGT, FLIPLGT, KLT, and a subset of elements of A, which is a reduced set of transform types, comprises subset x such that $x \in A$, and the subset x includes any of one or more transform types, {DCT, ADST, LGT, KLT}, and one or more combinations of vertical and horizontal transform types {DCT_DCT, LGT_LGT, DCT_LGT, LGT_DCT}; and

select an element of the subset x based on information that is separate from the video data, depending on any of an intra prediction mode and a block size, and by using, for directional intra prediction modes, only a nominal mode to select the element of the subset x, wherein the directional intra prediction modes share same nominal modes, including the nominal mode, and respectively comprise different angle delta values applied to a same implicit transform type.

12. The method of claim 11, further comprising considering one or more among 8 nominal modes, 5 non-angular smooth modes and an angle delta value during selection of the element of the subset x.

13. The method of claim 11, wherein selecting of the element of the subset x comprises any of:

selecting a same hybrid transform type for recursive-filtering modes and the DC mode, selecting a same hybrid transform type for the recursive-filtering modes and the SMOOTH mode,

selecting a same hybrid transform type for the SMOOTH mode, SMOOTH_H mode, and SMOOTH_V mode,

selecting a same hybrid transform type for the SMOOTH mode, SMOOTH_H mode, SMOOTH_V mode, and Paeth mode,

selecting a same hybrid transform type for the recursive-filtering modes, the SMOOTH mode, and Paeth mode,

selecting a same hybrid transform type for the V_PRED mode and SMOOTH_V mode,

selecting a same hybrid transform type for the H_PRED mode and the SMOOTH_H mode,

selecting a same hybrid transform type for the chroma-from-luma mode and the DC mode, or

selecting a same hybrid transform type for the chroma-from-luma mode and the SMOOTH mode.

14. The method of claim 11, wherein selecting the element of the subset x comprises any of:

selecting a same hybrid transform type for a chroma-from-luma mode and a Paeth mode,

using LGT, with different self-loop weights (v_{c1} , v_{c2}), depending on one or more among 8 nominal modes, 5 non-angular smooth modes, an angle delta value, a block size,

using KLTs, with different statistical properties, depending on one or more among 8 nominal modes, 5 non-angular smooth modes, an angle delta value, a block size, and

17

selecting a same hybrid transform type for a same intra prediction mode that is enabled for both luma and chroma components.

* * * * *

18